Design and Development of a Scalable Framework for Real-Time Multiplayer Gaming Using Unity and Networking

PHASE I REPORT

Submitted by

SUBASRI V	**230701201**
SUGANYA S	**230701351**
SUSHMA S	**230701355**
YASHVINTHINI R	**230701337**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

October 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "Design and Development of a Scalable Framework for Real-Time Multiplayer Gaming Using Unity and Networking" is the bonafide work of **SUBASRI V(230701346), SUGANYA S (230701351), SUSHMA S (230701355), YASHVINTHINI R (230701386)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates. "This project addresses the following Sustainable Development Goal(s): SDG 9 – Industry, Innovation and Infrastructure "

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Dr. P. Muneeshwari ,
Associate Professor,
Department of Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on

_____**Internal Examiner External Examiner**

ABSTRACT

An online multiplayer gaming platform is developed using Unity, Unity Gaming Services, and Netcode for GameObjects to enable real-time interaction between multiple players. The platform provides a common multiplayer infrastructure for user authentication, session creation and joining, networked player management, ownership handling, input processing, and synchronization. The system currently supports two game modules: a 2D multiplayer platformer and a 3D multiplayer racing game, both integrated within the same platform.

The 2D platformer supports networked player movement and synchronized multiplayer gameplay. The 3D racing module extends the platform with a dedicated racing environment, networked vehicle spawning, player-specific vehicle ownership, server-authoritative movement, and synchronized race management. Vehicle inputs are transmitted to the server, where movement and physics are processed before the updated vehicle positions and rotations are synchronized across connected clients. A server-controlled countdown ensures that all players begin the race simultaneously, while movement restrictions prevent vehicles from starting before the countdown is completed.

To maintain accurate race progression, the racing system implements ordered checkpoint validation and multi-lap tracking. Each vehicle maintains its individual checkpoint and lap state, ensuring that players progress independently and cannot skip required checkpoints. The current implementation supports three racing vehicles, three checkpoints per lap, and a five-lap race configuration.

The proposed modular architecture separates common multiplayer services from game-specific functionalities, allowing different game modes to share the same networking infrastructure. This reduces the need to redesign the complete multiplayer system when introducing additional games. The platform demonstrates how multiple multiplayer game types can be integrated using a reusable architecture while maintaining consistent player synchronization, ownership, and gameplay progression.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini,, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Dr. P. Muneeshwari, M.E.,Ph.D.,** Associate Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr.S.Madhusudhanan.,** Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

SUBASRI V 230701346
SUGANYA S 230701351
SUSHMA S 230701355
YASHVINTHINI R 230701386

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
	ABSTRACT	
	LIST OF TABLES	
	LIST OF FIGURES	
1.	**INTRODUCTION**	1
1.1	GENERAL	1
1.2	OBJECTIVE	1
1.3	EXISTING SYSTEM	2
1.4	PROPOSED SYSTEM	2
2.	**LITERATURE REVIEW**	3
	2.1 GENERAL	3
	2.2 INTRUSION DETECTION USING DEEP LEARNING	3
	2.3 ZERO-DAY AND UNSEEN ATTACK DETECTION	4
	2.4 FEATURE SELECTION AND REPRESENTATION ANALYSIS	5

2.5 SUMMARY OF THE REVIEWED WORK 6

2.6 RESEARCH GAP 7

3. SYSTEM DESIGN

7

3.1 SYSTEM ARCHITECTURE 9

3.2 SYSTEM REQUIREMENTS 9

3.2.1 SOFTWARE REQUIREMENTS 9

3.2.2 HARDWARE REQUIREMENTS

11

3.2.3 DATASET REQUIREMENTS 11

3.3 DATASET DESCRIPTION 12

3.4 DATA PREPROCESSING MODULE 13

3.5 FEATURE SELECTION MODULE 13

3.6 SEQUENCE CONSTRUCTION MODULE 14

3.7 CNN-GRU MODEL DESIGN
14

3.8 EVALUATION DESIGN 15
3.8.1 CLOSED-SET EVALUATION 15
3.8.2 LEAVE-ONE-ATTACK-OUT (LOAO) EVALUATION 15
3.8.3 DATA LEAKAGE AUDIT 16
3.8.4 FEATURE-SPACE ANALYSIS 16
3.9 EVALUATION METRICS 16

4. IMPLEMENTATION 17

4.1 IMPLEMENTATION ENVIRONMENT 17
4.2 DATA CLEANING 17

4.3 FEATURE SELECTION AND SCALING 18
4.4 SEQUENCE GENERATION AND MODEL TRAINING 18
4.5 LOAO IMPLEMENTATION 18
4.6 DATA LEAKAGE AUDIT 19
4.7 FEATURE-SPACE VISUALISATION 19
4.8 INTERACTIVE DEMONSTRATION 19
5. RESULTS AND DISCUSSION 19
5.1 FEATURE SELECTION RESULTS 19
5.2 CLOSED-SET CLASSIFICATION RESULTS 21
5.3 DATA LEAKAGE AUDIT FINDINGS 21
5.4 UNSEEN-ATTACK GENERALIZATION RESULT 21
5.5 FEATURE-SPACE ANALYSIS 24
5.6 DISCUSSION 25
5.7 LIMITATIONS 25
6. CONCLUSION AND FUTURE WORK 26
6.1 CONCLUSION 26
6.2 FUTURE WORK 26
REFERENCES

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Summary of the reviewed studies	6
3.1	Software Requirements of the proposed system	9
3.2	Hardware Requirements of the proposed system	11
3.3	Game Modes Implemented in the Proposed Platform	12
3.4	Evaluation metrics	16

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	System Architecture Diagram	8
5.1	Successful multiplayer session creation and session code generation.	34
5.2	Two networked players spawned in the same multiplayer environment.	35
5.3	Multiplayer lobby interface showing connected players before gameplay.	36
5.4	Leaderboard displaying player scores and rankings.	37
5.5	Networked racing vehicles spawned for connected players.	39
5.6	Server-authoritative vehicle movement and network synchronization.	40
5.7	Synchronized racing countdown before vehicle movement.	41
5.8	Multi-lap race progression with individual player race states.	42

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Online multiplayer gaming allows multiple players to interact and compete within a shared virtual environment. Unlike single-player games, multiplayer games require continuous communication and synchronization of player movements, actions, and game events to maintain consistent gameplay across connected systems. Developing such applications involves managing network connections, player ownership, sessions, and game-specific functionalities.

The proposed platform is developed using Unity, Unity Gaming Services, and Netcode for GameObjects to provide a common multiplayer infrastructure. It supports user authentication, session creation and joining, player spawning, ownership management, input handling, and real-time synchronization. This shared structure allows multiple game modes to operate within the same platform without requiring separate networking implementations.

The platform currently integrates two game modules: a 2D multiplayer platformer and a 3D multiplayer racing game. The platformer supports synchronized player movement and multiplayer interactions, while the racing module introduces networked vehicle spawning, individual vehicle ownership, server-authoritative movement, synchronized race countdown, checkpoint validation, and multi-lap progression. Vehicle movement is processed by the server and synchronized with connected clients to maintain consistent gameplay.

The proposed modular architecture separates common multiplayer services from game-specific functionalities, providing a reusable foundation for integrating

additional games. This approach simplifies future development while maintaining consistent player management, networking, and synchronization across different multiplayer game modes.

1.2 OBJECTIVE

In this project, the objective is to design a deep learning based system which detects and classifies different types of network traffic. The project includes data pre-processing and data quality analysis for training the model. Any invalid values, anomalies, duplicate feature vectors, constant features, and exact duplicate features are excluded from the dataset.

1.3 EXISTING SYSTEM

Existing multiplayer games are generally developed as individual applications with separate networking systems and game-specific functionalities. Each game requires its own implementation of player management, session creation, movement synchronization, and gameplay logic. This makes it difficult to reuse networking components across different game modes. Adding new games may also require additional development and modifications to the existing system. As a result, managing multiple multiplayer games within a single platform becomes complex, highlighting the need for a common and reusable multiplayer architecture.

1.4 PROPOSED SYSTEM

The proposed system aims to develop a unified online multiplayer gaming platform using Unity, Unity Gaming Services, and Netcode for GameObjects. Unlike separate multiplayer applications, the proposed platform provides a common networking infrastructure that supports multiple game modes within a single environment. It manages essential multiplayer functionalities such as user authentication, session

creation and joining, player spawning, ownership handling, input processing, and real-time synchronization.

The platform currently integrates two game modules: a 2D multiplayer platformer and a 3D multiplayer racing game. The 2D platformer allows multiple players to interact within a shared game environment with synchronized movement and gameplay actions. The 3D racing module introduces a dedicated racing environment with networked vehicle spawning, individual vehicle ownership, server-authoritative movement, synchronized race countdown, checkpoint validation, and multi-lap progression.

In the racing module, player inputs are transmitted to the server, where vehicle movement and physics are processed. The updated vehicle positions and rotations are then synchronized with other connected clients. A server-controlled countdown ensures that all players start the race simultaneously, while ordered checkpoints validate the racing path and prevent players from skipping required checkpoints. Each vehicle maintains its own checkpoint and lap information, allowing players to progress independently throughout the race.

The proposed architecture separates common multiplayer services from game-specific functionalities. Shared components such as authentication, session management, and network connections can be reused across different game modes, while individual games maintain their own gameplay logic and environments. This modular approach reduces repeated development efforts and simplifies the integration of additional multiplayer games in the future. It also provides a flexible foundation for expanding the platform with new game modes while maintaining consistent networking and synchronization.

3333CHAPTER 2
LITERATURE REVIEW

2.1 GENERAL

A considerable amount of research has been conducted on online multiplayer gaming, focusing on distributed game architectures, network synchronization, latency, consistency, fairness, and scalability. These studies explore different approaches to maintaining a consistent gaming experience across multiple connected players. To place the proposed project in context, the reviewed works are grouped into three main themes: distributed multiplayer architectures, synchronization and latency management, and scalability in networked games. The chapter concludes by identifying the research gap addressed by this project.

2.2 DISTRIBUTED MULTIPLAYER GAME ARCHITECTURES

Distributed architectures play an important role in multiplayer gaming by allowing multiple players to interact within a shared virtual environment. Gautier and Diot [1] proposed MiMaze, an Internet-based multiplayer game that used IP multicast communication through RTP/UDP/IP. Their system introduced distributed synchronization mechanisms to maintain consistency among participating players.

Bharambe, Pang, and Seshan [8] developed Colyseus, a distributed architecture designed to improve the scalability of online multiplayer games. Their approach utilized predictable game workloads and weakly consistent state updates to distribute network traffic and computation. The study demonstrated how distributed processing can support low-latency multiplayer gameplay.

Ahn et al. [7] proposed a real-time multiplayer game protocol architecture focusing on reducing network latency and improving server efficiency. Their approach

considered scheduling and spatial-domain techniques to manage client events and reduce communication delays.

These studies highlight the importance of selecting an appropriate networking architecture for supporting multiple players and maintaining consistent game interactions.

2.3 **NETWORK SYNCHRONIZATION AND LATENCY MANAGEMENT**

Synchronization is essential in multiplayer games because all connected players must receive consistent updates about the shared game environment. Lin, Guo, and Paul [2] proposed Sync-MS, a synchronized messaging service that addresses fairness issues caused by differences in network delays. Their approach used Sync-in and Sync-out mechanisms to coordinate player actions while balancing responsiveness and fairness.

Moon et al. [4] investigated consistency maintenance in distributed network games using conservative and optimistic synchronization approaches. Their study discussed the relationship between latency and consistency in multiplayer environments. Similarly, Mauve et al. [5] examined consistency in replicated applications and introduced local-lag and timewarp techniques to manage differences between replicated game states.

Pantel and Wolf [6] studied the impact of network transmission delays on real-time multiplayer games and highlighted how communication delays can create differences in the game state perceived by players. Liu et al. [10] examined the effects of network

latency on competitive first-person shooter players and discussed how latency compensation can reduce its negative impact on gameplay.

These studies demonstrate that synchronization and latency management are important for maintaining responsive and consistent multiplayer interactions.

2.4 SCALABILITY AND NETWORK PERFORMANCE

Scalability is another important consideration in multiplayer game development, particularly when the number of connected players and network traffic increases. Jiang, Safaei, and Boustead [3] reviewed major challenges associated with latency and scalability in networked games. Their work discussed techniques for handling increasing player populations and communication requirements.

Fujita [9] investigated load balancing in peer-to-peer massively multiplayer online role-playing games using hierarchical area-of-interest management. The approach dynamically adjusted resource allocation based on player distribution across different game regions.

The reviewed studies indicate that network performance, resource management, and load distribution influence the ability of multiplayer systems to support increasing numbers of players. These considerations are relevant when designing reusable multiplayer infrastructure for different game environments.

2.5 SUMMARY OF THE REVIEWED WORK

Table 2.1 Summary of the reviewed studies

Ref.	Focus of the study	Relevance to this project
[1]	MiMaze distributed multiplayer game using IP multicast	Demonstrates distributed communication and

		synchronization in multiplayer games.
[2]	Sync-MS synchronized messaging service	Highlights synchronization and fairness in multiplayer interactions.
[3]	Survey of latency and scalability challenges in networked games	Identifies important networking challenges in multiplayer environments.
[4]	Consistency maintenance using conservative and optimistic approaches	Supports the importance of maintaining consistent game states.
[5]	Consistency in replicated continuous applications	Provides approaches for handling synchronization delays.
[6]	Impact of network transmission delay on multiplayer games	Highlights the effect of latency on player experience.
[7]	Real-time multiplayer protocol architecture	Supports efficient handling of client events and network communication.
[8]	Colyseus distributed multiplayer architecture	Demonstrates distributed processing and scalability in online games.

[9]	Load balancing using hierarchical area-of-interest management	Highlights resource allocation in large multiplayer environments.
[10]	Effects of network latency on competitive FPS players	Demonstrates the importance of latency reduction and compensation.

2.6 RESEARCH GAP

The reviewed studies primarily focus on individual networking challenges such as synchronization, latency reduction, consistency, and scalability. Although these approaches provide useful solutions for distributed multiplayer environments, the integration of common multiplayer services with multiple game modules remains an important development consideration.

This project addresses this gap by developing a modular multiplayer gaming platform using Unity, Unity Gaming Services, and Netcode for GameObjects. The platform integrates a 2D multiplayer platformer and a 3D multiplayer racing game using shared authentication, session management, player ownership, and networking functionalities. The racing module further implements server-authoritative vehicle movement, synchronized race countdown, ordered checkpoint validation, and multi-lap progression. The proposed system demonstrates how different multiplayer game modes can operate using a common networking infrastructure without redesigning the complete system for each game.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The proposed system is a real-time multiplayer gaming platform developed using Unity and Unity Gaming Services. Its architecture provides a common multiplayer framework through which players can authenticate, create or join sessions, enter a lobby, and participate in different game modes. The current implementation includes a 2D multiplayer platformer and a 3D multiplayer racing game.

The architecture begins with the initialization of Unity Gaming Services, followed by player authentication. After authentication, a player can create a multiplayer session and act as the host, while other players can join the session using the session code. The session management component establishes the connection between participants and provides the foundation for multiplayer gameplay.

Netcode for GameObjects is used to manage networked objects and communication between connected players. The NetworkManager coordinates network connections and networked object management. When players join a session, their corresponding player objects or racing vehicles are spawned in the game environment. Ownership is assigned to associate each networked object with its controlling player.

The common multiplayer layer is shared by the two game modes, while their gameplay operations are handled separately. In the 2D platformer, the system manages player input, character movement, synchronization, elimination, and leaderboard functionality. In the 3D racing module, the system manages vehicle

spawning, ownership, server-authoritative vehicle movement, race countdown, checkpoint validation, and lap progression.

The overall architecture therefore separates multiplayer connectivity from game-specific operations. This structure allows the existing multiplayer services and networking components to support the developed game modes without requiring each mode to implement a separate connection workflow.

The main stages of the proposed architecture are:
Unity Project Initialization → UGS Initialization → Player Authentication → Session Creation or Joining → Lobby → Network Connection → Networked Object Spawning → Ownership Assignment → Game Mode Selection →
Multiplayer Gameplay → Game-Specific State Management → Network Synchronization → Evaluation.

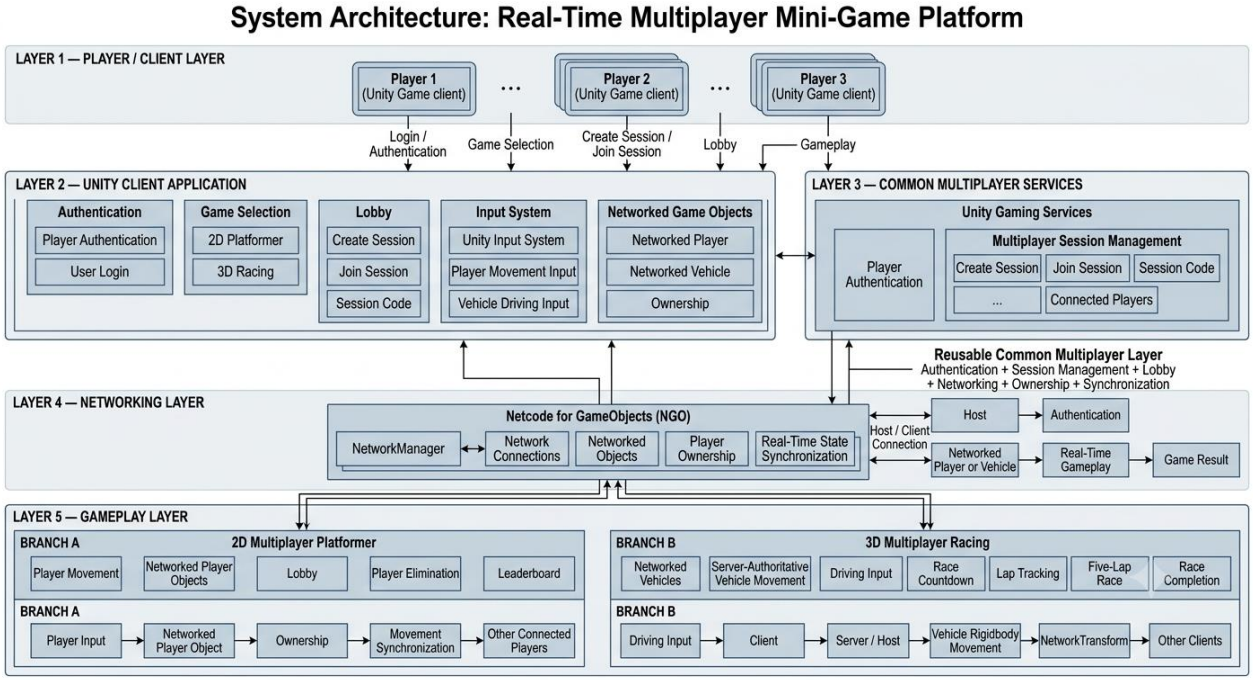


Figure 3.1 Architecture System Flow Diagram.

For the 3D racing mode, the gameplay stage includes client input transmission, server-side vehicle movement, synchronized countdown, ordered checkpoint validation, and multi-lap progression.

3.2 SYSTEM REQUIREMENTS

3.2.1 SOFTWARE REQUIREMENTS

Table 3.1 Software Requirements of the Proposed System

S. No.	Software / Tool	Purpose
1	Unity	Primary game development environment used to create the 2D and 3D game environments.
2	C#	Programming language used to implement gameplay logic, player control, networking, and race management.
3	Unity Gaming Services (UGS)	Provides the multiplayer-related services used for authentication and session management.
4	Netcode for GameObjects	Manages networked game objects and multiplayer communication.
5	NetworkManager	Coordinates multiplayer connections and networked object management.

6	Unity Input System	Captures player input for character movement and vehicle control.
7	NetworkObject	Enables player objects and racing vehicles to participate in the networked environment.
8	NetworkTransform	Synchronizes the position and rotation of networked racing vehicles.
9	Unity Multiplayer Play Mode / Multiple Game Instances	Supports testing of multiplayer functionality using connected game instances.
10	Unity Editor and Project Assets	Used for scene development, configuration, debugging, and gameplay testing.

3.2.2 HARDWARE REQUIREMENTS

Table 3.2 Hardware Requirements of the Proposed System

S. No.	Hardware Component	Requirement / Purpose
1	Processor	A multi-core processor suitable for running the Unity Editor and game instances.

2	RAM	Sufficient memory to support Unity development, scene execution, and multiplayer testing.
3	Storage	Adequate storage for the Unity project, game assets, scenes, and development files.
4	GPU	Supports rendering of the 2D platformer, 3D racing environment, and Unity Editor interface.
5	Display	Required for game development, scene editing, debugging, and system demonstration.
6	Keyboard & Mouse	Used for Unity Editor interaction and testing player input.
7	Network Connection	Required for connecting game instances through the multiplayer services.

3.2.3 GAME DATA AND ASSET REQUIREMENTS

The proposed platform does not depend on a fixed machine-learning dataset. Instead, it uses game assets, networked objects, player input, and gameplay state information during execution.

The 2D platformer requires a game environment containing ground areas, platforms, player objects, camera components, lighting, and multiplayer interface elements. It also uses player-related gameplay information for movement, elimination, and leaderboard operations.

The 3D racing module requires a racing scene containing the track and its environment objects, including roads, barriers, fences, cones, tyres, finish-line structures, signs, stands, lighting, and other visual elements. Three predefined vehicle spawn positions are provided for the connected players. Each vehicle maintains its own checkpoint and lap progression during the race.

The multiplayer system also handles runtime information such as player ownership, networked object state, movement input, vehicle state, checkpoint progress, and lap count. These values are used by the respective game modules during multiplayer execution.

3.3 DESCRIPTION OF THE DEVELOPED GAME MODES

The proposed platform currently supports two multiplayer game modes. Both use the common authentication, session management, and networking framework, while their gameplay logic differs according to the requirements of each game.

Table 3.3 Game Modes Implemented in the Proposed Platform

Game Mode	Gameplay Type	Main Implemented Features
2D Multiplayer Platformer	Two-dimensional character-based gameplay	Networked player spawning, ownership, input handling, movement synchronization, lobby, elimination, and leaderboard.
3D Multiplayer Racing	Three-dimensional vehicle-based gameplay	Networked vehicle spawning, vehicle ownership, server-authoritative movement, synchronized countdown, ordered checkpoints, and five-lap progression.

The 2D platformer provides the initial game environment for validating the common multiplayer framework. The 3D racing module extends the platform by demonstrating how the same multiplayer foundation can support a different gameplay structure involving vehicle physics and race progression.

3.4 MULTIPLAYER INITIALIZATION AND AUTHENTICATION MODULE

The initialization module prepares Unity Gaming Services for multiplayer operations when the application starts. Player authentication is performed after service initialization to provide an identity for each participant.

Authentication is part of the multiplayer connection workflow because players must be identified before participating in a session. Once authentication is completed, the player can proceed to create or join a multiplayer session.

This module provides the initial stage of the common multiplayer workflow and is used by both the 2D platformer and the 3D racing game.

3.5 SESSION MANAGEMENT AND LOBBY MODULE

The session management module allows a player to create a multiplayer session or join an existing session using a session code. The player who creates the session acts as the host, while other participants join as clients.

The session code provides a mechanism for connecting players to the same multiplayer environment without requiring manual network configuration. After joining, players can enter the implemented multiplayer lobby before starting gameplay.

The lobby provides an intermediate stage between session connection and gameplay. It allows connected participants to be represented within the same session and provides an organized transition into the selected game mode.

The session and lobby workflow can be represented as:
Player Authentication → Session Creation / Joining → Player Connection → Lobby → Game Mode Entry

3.6 NETWORKED PLAYER AND OWNERSHIP MODULE

Netcode for GameObjects is used to manage networked player objects within the multiplayer environment. When a player successfully joins a session, the corresponding player object is spawned.

The NetworkObject component allows a player object to participate in the networked environment. Ownership associates the object with the player responsible for

controlling it. This distinction is important because each participant should control their own player object, while remote player objects remain visible to other connected participants.

The same ownership concept is applied to the racing vehicles. Each networked vehicle is associated with its corresponding player, allowing the system to distinguish local vehicle control from the representation of other players' vehicles.

3.7 INPUT AND MOVEMENT MODULE

The player input module uses Unity's Input System to capture input from the local player. In the 2D platformer, movement actions and Vector2 input are used to control the corresponding networked character.

The input configuration is connected to the player's networked object through the ownership mechanism. This ensures that input is applied to the appropriate player character rather than to a remote player's object.

The module was tested during development by examining Input Actions, movement bindings, player ownership, spawning, and the relationship between the local player and its networked object. The resulting movement state is made available to other connected participants through the multiplayer networking system.

3.8 2D MULTIPLAYER GAMEPLAY MODULE

The 2D multiplayer platformer is the initial game mode developed using the proposed framework. Its environment includes a ground area, multiple platforms, networked player objects, camera components, lighting, and multiplayer interface elements.

The game environment is used to test the common multiplayer functionality, including player spawning, ownership, input handling, and movement synchronization. Multiple connected players can participate in the same environment and control their respective characters.

The platformer also includes a player elimination system and leaderboard functionality. The elimination system updates the gameplay state when the defined elimination condition is reached. The leaderboard displays and compares player scores based on their performance during gameplay.

These components demonstrate how game-specific operations can be integrated with the common multiplayer framework.

3.9 3D MULTIPLAYER RACING MODULE

The 3D racing module is implemented in a dedicated Unity scene using Netcode for GameObjects. A RaceManager is responsible for initializing the race and creating networked racing vehicles for connected players.

Three predefined vehicle spawn positions are provided so that players can begin from separate positions on the track. Each vehicle is configured as a NetworkObject, and ownership is assigned to the corresponding player.

Vehicle control follows a server-authoritative model. The local client collects driving input and sends the input values to the server through RPC communication. The server processes the received input and applies it to the vehicle Rigidbody. NetworkTransform synchronizes the resulting vehicle position and rotation across connected clients.
The vehicle supports forward and reverse movement through throttle input and left and right steering. A maximum speed is applied to control vehicle movement. Rigidbody physics is used for movement and collision handling, while the server maintains authority over the vehicle's physical state.

A third-person follow camera is provided for each player. It identifies the locally owned racing vehicle and follows it using a configured offset and smooth movement.

3.9.1 Race Countdown Module

A synchronized countdown is used before the race begins. The server controls the countdown sequence of 3, 2, 1, and GO. Vehicle movement remains disabled during the countdown. When the GO state is reached, the race starts and vehicle input is enabled.

This process provides a synchronized starting stage for the connected players.

3.9.2 Checkpoint Validation Module

The racing track contains checkpoints implemented using trigger colliders. Each checkpoint is assigned a unique number, and the system maintains the next expected checkpoint for each vehicle.

The checkpoints must be passed in the defined order. For example, the expected sequence is CP1 → CP2 → CP3. If a vehicle reaches CP3 before completing CP2, its race progress is not advanced. This prevents incorrect checkpoint order from being counted as valid progress.

3.9.3 Lap Progression Module

Three checkpoints form one complete lap. After a vehicle successfully passes CP1, CP2, and CP3, its next expected checkpoint is reset to CP1 and its lap count is increased.

Each vehicle maintains its own current lap, next expected checkpoint, and race progress. This allows different players to progress independently during the same race.

The racing module is configured for five laps, and the race management system identifies completion after the required lap progression is reached.

3.10 NETWORK SYNCHRONIZATION MODULE

Network synchronization allows connected participants to observe relevant changes in the shared multiplayer environment. In the 2D platformer, networked player objects are managed through Netcode for GameObjects, with ownership distinguishing local control from remote player representation.

In the racing module, client input is transmitted to the server through RPC communication. The server processes vehicle movement and updates the authoritative vehicle state. NetworkTransform then synchronizes vehicle position and rotation with connected clients.

The synchronization mechanisms differ according to the gameplay requirements of each mode. The common networking framework manages connected participants and networked objects, while each game mode handles its own gameplay state.

3.11 DEVELOPMENT AND DEBUGGING MODULE

The development process involved testing the interaction between Unity Input System, networked objects, player ownership, session management, player spawning, lobby management, and gameplay components.

During 2D player testing, input configuration and ownership were examined when a player's object was visible but did not initially respond to the expected local input. The Input Actions, Vector2 movement bindings, ownership assignment, and networked player setup were checked to improve the control behavior.

The racing module was tested by examining vehicle spawning, ownership, client input transmission, server-side Rigidbody movement, NetworkTransform synchronization, countdown behavior, checkpoint ordering, and lap progression.

Session lifecycle behavior was also examined to avoid performing session-related operations when a corresponding multiplayer session was not active. These activities

helped improve the integration of the individual components within the multiplayer prototype.

3.12 OVERALL DEVELOPMENT WORKFLOW

The complete development workflow combines common multiplayer operations with the gameplay-specific operations of the two implemented game modes.

Unity Project Initialization → UGS Initialization → Player Authentication → Session Creation / Joining → Lobby → Network Connection → Networked Object Spawning → Ownership Assignment → Game Mode Entry → Gameplay Input → Game-Specific Processing → Network Synchronization → Multiplayer Gameplay → Evaluation

For the 2D platformer, the gameplay-specific operations include character movement, elimination, score updates, and leaderboard display.
For the 3D racing mode, the workflow includes vehicle spawning, ownership assignment, client input transmission, server-authoritative vehicle movement, synchronized countdown, checkpoint validation, and multi-lap progression.

The proposed system therefore combines a common multiplayer connection framework with separate gameplay modules. This arrangement supports the two developed game modes while allowing the multiplayer architecture to be extended with additional games in future development.

3.13 EVALUATION DESIGN

The developed platform was evaluated to verify the operation of its common multiplayer components and the gameplay functionality of both implemented game modes. Testing was performed using multiple game instances connected to the same multiplayer session.

3.13.1 Authentication and Session Evaluation

The authentication and session workflow was tested by initializing the multiplayer services, authenticating a player, and creating a multiplayer session. The generated session code was then used by another player to join the existing session.
The evaluation examined whether the session could be created and whether another participant could connect using the provided code.

3.13.2 Networked Player and Ownership Evaluation

After the client joined the session, the corresponding player object was checked in the shared 2D game environment. The evaluation focused on player spawning, ownership assignment, and whether each participant controlled the appropriate local character.
Movement was tested using connected game instances to verify that each player could control their own character while observing the other player's networked movement.

3.13.3 Lobby, Elimination, and Leaderboard Evaluation

The lobby was tested to verify that connected players could be represented before entering gameplay. The elimination system was examined to confirm that the

appropriate player's gameplay state was updated when the defined elimination condition occurred.

The leaderboard was tested to verify that player scores were displayed and compared according to their gameplay performance.

3.13.4 Racing Vehicle and Movement Evaluation

The racing module was evaluated using connected game instances. Testing covered the creation of networked vehicles at the predefined spawn positions, ownership assignment, and vehicle input.

Forward and reverse movement, steering, maximum speed, and Rigidbody-based movement were examined. The evaluation also checked that client input was sent to the server through RPC communication and that the resulting vehicle position and rotation were synchronized using NetworkTransform.

3.13.5 Race Countdown and Progression Evaluation

The race countdown was tested to verify the sequence of 3, 2, 1, and GO. Vehicle input remained disabled during the countdown and became enabled after the GO state. Checkpoint validation was tested using the sequence CP1 → CP2 → CP3. The evaluation checked whether progress advanced only when the expected checkpoint was reached. Lap progression was then examined to verify that completing all three checkpoints increased the vehicle's lap count and reset its next expected checkpoint to CP1.

The racing module was configured for five laps, and race completion detection was examined after the required lap progression.

3.13.6 Complete Multiplayer Workflow Evaluation

The complete workflow was examined from authentication and session joining through lobby participation and multiplayer gameplay. For the platformer, testing included player spawning, ownership, movement synchronization, elimination, and leaderboard functionality. For racing, testing included vehicle spawning, server-authoritative movement, countdown, checkpoint validation, and lap progression.
The evaluation focused on the integration of the common multiplayer framework with the game-specific operations of both modes. Final finishing-position calculation, racing leaderboard integration, car selection, and track selection are outside the current implementation.

3.14 EVALUATION PARAMETERS

The proposed system is evaluated through functional and gameplay-related checks rather than classification metrics. The following parameters describe the main operations examined during testing.

Table 3.4 Evaluation metrics

Metric	Evaluation Purpose
Authentication	Checks whether a player can complete the authentication stage before entering a multiplayer session.

Session creation	Checks whether a host can create a multiplayer session and obtain a session code.
Session joining	Checks whether another player can join the existing session using the session code.
Player spawning	Checks whether connected players are represented by networked objects in the shared environment.
Ownership	Checks whether each player controls the corresponding owned player object or vehicle.
Movement synchronization	Checks whether player movement is visible across connected game instances.
Lobby functionality	Checks whether connected participants can be managed before gameplay begins.
Elimination	Checks whether the defined elimination condition updates the appropriate player's gameplay state.
Leaderboard	Checks whether player scores and rankings are displayed during platformer gameplay.
Vehicle movement	Checks forward, reverse, and steering input with server-side Rigidbody processing.
Vehicle synchronization	Checks whether vehicle position and rotation are synchronized across connected clients.

Race countdown	Checks the synchronized 3, 2, 1, GO sequence and movement lock before the race begins.
Checkpoint validation	Checks whether vehicles progress only through the expected checkpoint sequence.
Lap progression	Checks whether three completed checkpoints increase the lap count and reset the next expected checkpoint.
Race completion	Checks whether completion is detected after the configured five-lap progression.

These parameters provide a functional basis for evaluating the implemented multiplayer platform. They cover both the shared networking workflow and the distinct gameplay operations of the 2D platformer and 3D racing module.

CHAPTER 4

IMPLEMENTATION

4.1 IMPLEMENTATION ENVIRONMENT

The proposed multiplayer gaming platform was developed using Unity and C#. Unity was used to create the game environments, configure scenes, and integrate the gameplay components. Unity Gaming Services was used for multiplayer-related services, including player authentication and session management. Netcode for GameObjects was used to manage networked objects and multiplayer

communication, while NetworkManager coordinated the network connections and networked object management.

Unity Input System was used to capture player input in the 2D platformer and the 3D racing module. The 2D game includes networked player movement, elimination, and leaderboard functionality. The racing module uses RPC communication to send driving input to the server, where vehicle movement is processed using Rigidbody physics. NetworkTransform is used to synchronize vehicle position and rotation across connected clients.

The implementation was tested using multiple game instances connected to the same multiplayer session. The development and debugging process involved examining service initialization, authentication, session creation and joining, player spawning, ownership, input handling, movement synchronization, lobby functionality, elimination, leaderboard operations, and racing progression.

4.2 UNITY PROJECT AND SCENE IMPLEMENTATION

The project was developed in Unity with separate game environments for the 2D multiplayer platformer and the 3D multiplayer racing module. The common multiplayer components provide the connection and networking foundation, while each scene contains the components required for its respective gameplay.

The 2D platformer scene includes the ground area, platforms, player objects, camera components, lighting, and multiplayer user interface. The racing scene contains the 3D track, roads, barriers, fences, cones, tyres, finish-line structures, signs, stands, lighting, and other environment objects.

A dedicated RaceManager is used in the racing scene to initialize the race and create networked vehicles for connected players. Three predefined vehicle spawn positions are configured to allow players to begin from separate locations on the track.

4.3 MULTIPLAYER SERVICES INITIALIZATION AND AUTHENTICATION

Unity Gaming Services is initialized when the application starts. Player authentication is then performed before the player proceeds to multiplayer session operations.
The authentication process is integrated into the common multiplayer workflow. After successful initialization and authentication, a player can create a new session or join an existing session using its session code. This workflow is shared by the two implemented game modes.

4.4 SESSION CREATION AND JOINING IMPLEMENTATION

The session management functionality was implemented using Unity Multiplayer Services. A player can create a multiplayer session and act as the host. Other players can join the same session by entering the session code generated for that session.
The joining process connects participants to the same multiplayer environment without requiring manual network configuration. After joining, players can enter the multiplayer lobby before starting gameplay.
The session workflow was implemented as: **Player Authentication → Session Creation / Joining → Player Connection → Lobby → Game Mode Entry**
This common connection workflow provides the foundation for both the platformer and racing modules.

4.5 NETWORKED PLAYER SPAWNING AND OWNERSHIP

Netcode for GameObjects was used to manage networked player objects. When a player successfully joins the session, the corresponding player object is spawned in the game environment.

The NetworkObject component allows the player object to participate in the networked environment. Ownership is assigned to associate the object with the player responsible for controlling it. This allows the system to distinguish the locally controlled player from remote player representations.

During development, player spawning and ownership were tested together because a player object could be visible in another game instance without necessarily receiving the expected local input. The Input System configuration, ownership assignment, and networked player setup were examined to resolve the control behavior.

4.6 2D PLAYER INPUT AND MOVEMENT IMPLEMENTATION

Player movement in the 2D platformer was implemented using Unity's Input System. Movement actions and Vector2 input are used to capture the local player's movement commands.

The input is applied to the corresponding networked player object based on its ownership. The networking system allows other connected participants to observe the player's movement in the shared game environment.

During implementation, the Input Actions and movement bindings were checked to ensure that the correct input was received by the locally owned player object. The relationship between player ownership, spawning, and input handling was also examined during debugging.

4.7 2D MULTIPLAYER GAMEPLAY IMPLEMENTATION

The 2D multiplayer platformer was implemented as the initial game mode of the proposed platform. Multiple connected players can enter the same environment and control their respective networked characters.

The platformer includes a lobby system, player elimination functionality, and a leaderboard. The lobby manages connected participants before gameplay begins. The elimination system updates a player's gameplay state when the defined elimination condition is reached. The leaderboard displays and compares player scores based on their performance. These gameplay components were integrated with the common multiplayer framework to demonstrate the use of shared networking services within an interactive game.

4.8 LOBBY, ELIMINATION, AND LEADERBOARD IMPLEMENTATION

The multiplayer lobby provides an intermediate stage between session joining and gameplay. After authentication and session creation or joining, players can enter the lobby and be represented as participants in the same session.

The elimination system monitors the defined gameplay condition and updates the corresponding player's state when elimination occurs. The updated state is reflected in the multiplayer environment.

The leaderboard maintains and displays player scores to provide a comparison of performance during gameplay. It is integrated with the platformer scoring functionality and displays the resulting player rankings.
These components were tested as part of the overall multiplayer workflow.

4.9 3D RACING VEHICLE IMPLEMENTATION

The 3D racing module was implemented in a dedicated Unity scene. The RaceManager initializes the race and creates networked vehicles for connected players. Three predefined spawn positions are available, allowing vehicles to begin from separate positions on the track.

Each racing vehicle is configured as a NetworkObject, and ownership is assigned to the corresponding player. This associates each vehicle with its controlling participant. The vehicle supports forward and reverse movement through throttle input and left and right steering. A maximum speed is applied to control vehicle movement. Rigidbody physics is used for vehicle movement and collision handling.

A third-person follow camera is provided for each player. It identifies the locally owned vehicle and follows it using a configured offset and smooth movement.

4.10 SERVER-AUTHORITATIVE VEHICLE MOVEMENT

The racing module follows a server-authoritative vehicle control model. The local client collects driving input and sends the input values to the server through RPC communication.

The server processes the received input and applies it to the vehicle Rigidbody. The resulting vehicle position and rotation are synchronized across connected clients using NetworkTransform.
This implementation separates input collection on the client from the processing of the vehicle's physical state on the server. It allows connected players to observe the movement of the networked vehicles within the same racing environment.

4.11 RACE COUNTDOWN IMPLEMENTATION

A synchronized countdown was implemented before the racing gameplay begins. The server controls the countdown sequence of 3, 2, 1, and GO.

Vehicle movement remains disabled while the countdown is active. After the GO state is reached, the race is marked as started and vehicle input is enabled.

The countdown is managed through the race state so that connected players participate in the race from the synchronized starting stage.

4.12 CHECKPOINT AND LAP PROGRESSION IMPLEMENTATION

The racing track contains checkpoints implemented using trigger colliders. Each checkpoint is assigned a unique number, and the system maintains the next expected checkpoint for each vehicle.

The checkpoints must be passed in the specified order: CP1 → CP2 → CP3. If a vehicle reaches a later checkpoint before completing the required checkpoint, its

progress is not advanced. Repeatedly triggering the same checkpoint does not incorrectly increase the player's progress.
Three checkpoints form one complete lap. After successfully passing CP1, CP2, and CP3, the vehicle's next expected checkpoint is reset to CP1 and its lap count is increased.

Each vehicle maintains its own current lap, next expected checkpoint, and race progress. This allows connected players to progress independently during the same race. The racing module is configured for five laps, and race completion is detected after the required lap progression is reached.

4.13 DEVELOPMENT AND DEBUGGING

The development process involved testing the interaction between Unity Input System, networked objects, player ownership, session management, player spawning, lobby management, and gameplay components.

In the 2D platformer, input configuration and player ownership were examined when a networked player object was visible but did not initially respond to the expected local input. Input Actions, Vector2 movement bindings, ownership assignment, and the networked player setup were checked to improve the control behavior.

In the racing module, testing focused on vehicle spawning, ownership, client input transmission, server-side Rigidbody movement, NetworkTransform synchronization, countdown behavior, checkpoint ordering, and lap progression.

Session lifecycle behavior was also examined to avoid performing session-related operations when a corresponding multiplayer session was not active. These activities

helped improve the integration of the different components within the multiplayer prototype.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 MULTIPLAYER SESSION CREATION RESULTS

The session creation functionality was tested by initializing Unity Gaming Services, authenticating the player, and creating a multiplayer session. The host was able to create a session and obtain a session code that could be shared with another player. The session code provided a simple method for connecting participants to the same multiplayer environment without requiring manual network configuration.

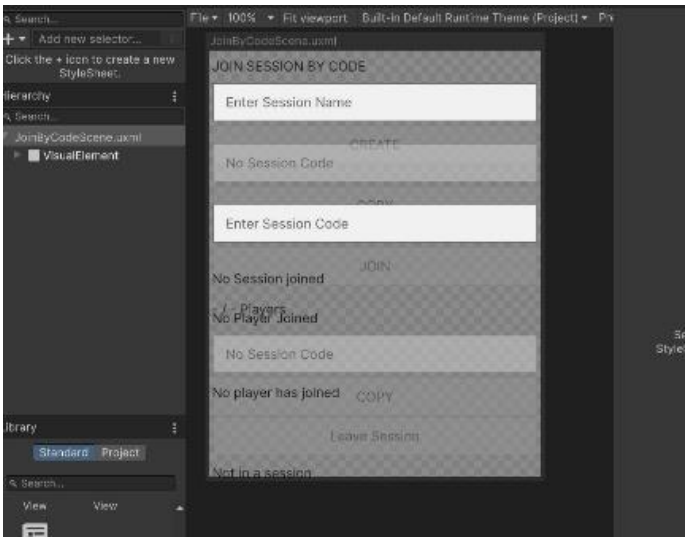


Figure 5.1 Successful multiplayer session creation and session code generation.

5.2 SESSION JOINING RESULTS

The session joining functionality was tested using a second game instance. The client entered the session code generated by the host and joined the existing multiplayer session.

The successful joining process demonstrated that the session management component could connect the host and client to the same multiplayer environment. This established the connection required for subsequent player spawning and gameplay tests.

5.3 NETWORKED PLAYER SPAWNING RESULTS

After the client joined the session, the corresponding networked player object was spawned in the 2D platformer environment. Both the host and client could observe the connected players in the same scene.

The result demonstrated the integration of session management and networked object spawning. It also confirmed that the connected participants were represented within the shared multiplayer environment.

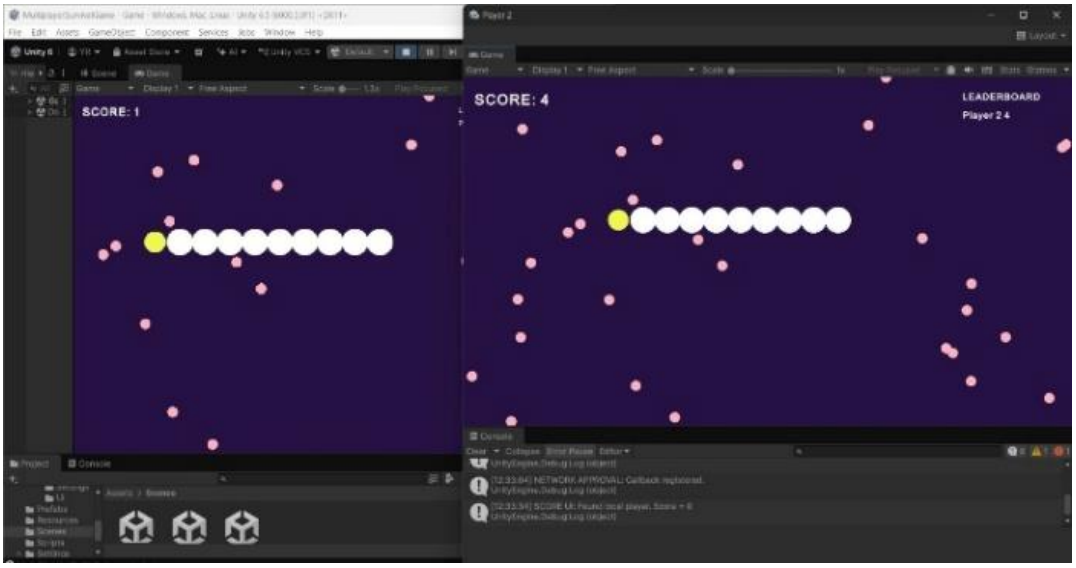


Figure 5.2 Two networked players spawned in the same multiplayer environment.

5.4 PLAYER MOVEMENT AND OWNERSHIP RESULTS

Player movement was tested using two connected game instances. Each player was able to control the corresponding local player character, while the movement of the other player was visible in the connected instance.

The testing demonstrated the interaction between Unity Input System, player ownership, networked player objects, and Netcode for GameObjects. Ownership was important for distinguishing local player control from remote player representation.
During development, an issue was observed in which the second player's object was visible but did not initially respond to the expected input. Investigation of the Input System configuration, ownership, and networked player setup helped improve the control behavior.

5.5 MULTIPLAYER LOBBY RESULTS

The lobby system was tested to verify that connected players could be managed before entering gameplay. After authentication and session creation or joining, participants could enter the lobby and be represented within the same multiplayer session.
The lobby provided an intermediate stage between session connection and gameplay. The test demonstrated that connected participants could be organized before starting the selected game mode.



Figure 5.3 Multiplayer lobby interface showing connected players before gameplay.

5.6 PLAYER ELIMINATION RESULTS

The player elimination system was tested as part of the 2D multiplayer gameplay. The system updated the appropriate player's gameplay state when the defined elimination condition was reached.

The elimination behavior was observed in the multiplayer environment to verify that the corresponding state change was reflected during gameplay. The test demonstrated that the elimination mechanism could be integrated with the networked player architecture.

5.7 LEADERBOARD RESULTS

The leaderboard functionality was tested to verify the display and comparison of player performance. Player scores were recorded and displayed according to their gameplay performance.

The leaderboard provided a common view of player rankings during multiplayer gameplay. This demonstrated the integration of scoring and performance comparison within the 2D platformer.

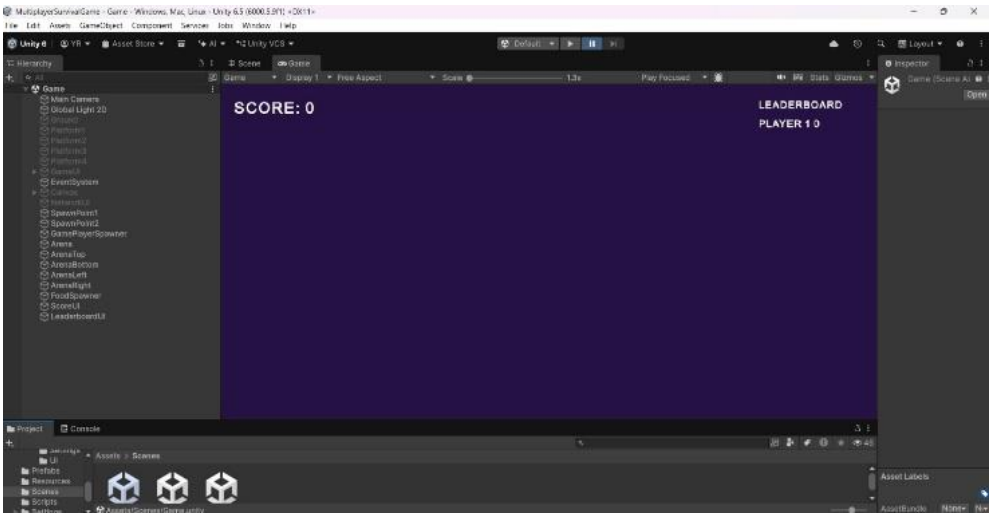


Figure 5.4 Leaderboard displaying player scores and rankings.

5.8 COMPLETE 2D MULTIPLAYER GAMEPLAY RESULTS

The complete 2D multiplayer workflow was tested by running two players in the same session. The host created the session, and the client joined using the session code. Both participants entered the multiplayer environment and controlled their corresponding player characters.

The gameplay test included networked player spawning, ownership, movement synchronization, elimination, and leaderboard functionality. The result demonstrated that the common multiplayer components and the platformer-specific gameplay operations could work together within the same session.

5.9 3D RACING ENVIRONMENT AND VEHICLE SPAWNING RESULTS

The racing scene was tested to verify the initialization of the 3D environment and the creation of networked vehicles for connected players. The scene contains the racing track, road boundaries, barriers, cones, finish-line structures, signs, lighting, and other environment objects.

The RaceManager creates vehicles for connected players using three predefined spawn positions. Each vehicle is configured as a NetworkObject and is associated with the corresponding player.

The test demonstrated the integration of the racing scene, vehicle spawning, and player ownership within the multiplayer environment.



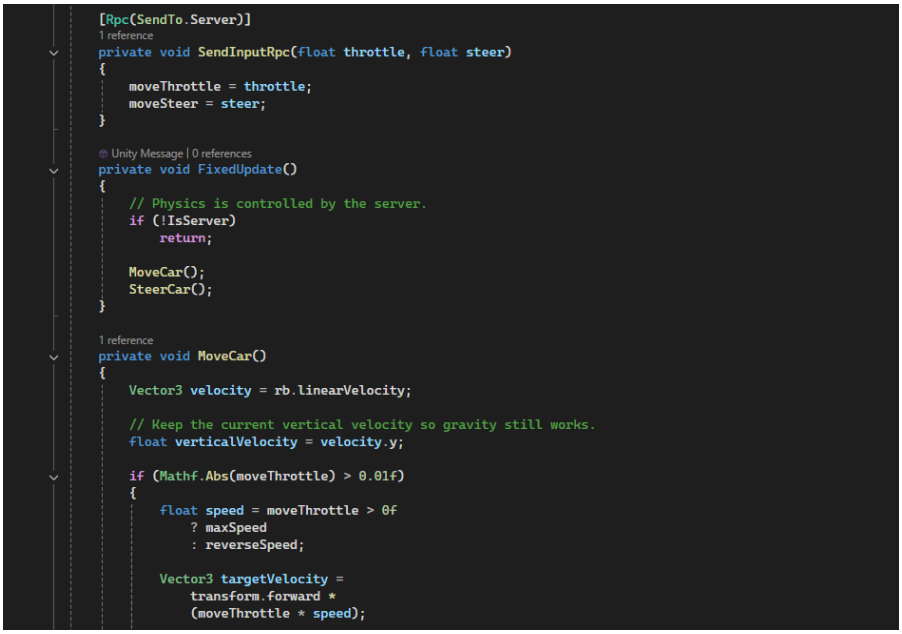
Figure 5.5 Networked racing vehicles spawned for connected players.

5.10 VEHICLE OWNERSHIP AND MOVEMENT RESULTS

Vehicle ownership was tested to verify that each player could control the racing vehicle assigned to them. The locally owned vehicle received the player's driving input, while other vehicles remained visible as networked objects.

Forward and reverse movement, steering, maximum speed, and Rigidbody-based movement were examined. The client transmitted driving input to the server through RPC communication, and the server processed the input to update the vehicle's physical state.

NetworkTransform synchronized the resulting vehicle position and rotation with connected clients. The test demonstrated the integration of client input transmission, server-side vehicle processing, and network synchronization.



```
[Rpc(SendTo.Server)]
1 reference
private void SendInputRpc(float throttle, float steer)
{
    moveThrottle = throttle;
    moveSteer = steer;
}

Unity Message | 0 references
private void FixedUpdate()
{
    // Physics is controlled by the server.
    if (!IsServer)
        return;

    MoveCar();
    SteerCar();
}

1 reference
private void MoveCar()
{
    Vector3 velocity = rb.linearVelocity;

    // Keep the current vertical velocity so gravity still works.
    float verticalVelocity = velocity.y;

    if (Mathf.Abs(moveThrottle) > 0.01f)
    {
        float speed = moveThrottle > 0f
            ? maxSpeed
            : reverseSpeed;

        Vector3 targetVelocity =
            transform.forward *
            (moveThrottle * speed);
```

Figure 5.6 Server-authoritative vehicle movement and network synchronization.

5.11 RACE COUNTDOWN RESULTS

The racing countdown was tested to verify the sequence of 3, 2, 1, and GO. The server controlled the countdown, and vehicle movement remained disabled while the countdown was active.

After the GO state was reached, the race started and vehicle input was enabled. This demonstrated the implementation of the synchronized race-start mechanism.



Figure 5.7 Synchronized racing countdown before vehicle movement.

5.12 CHECKPOINT VALIDATION RESULTS

The checkpoint system was tested using the expected sequence CP1 → CP2 → CP3. Each checkpoint was identified by a unique number and implemented using a trigger collider.

The evaluation checked whether a vehicle advanced only after reaching its expected checkpoint. If a later checkpoint was triggered before the required checkpoint, the player's progress was not advanced. Repeatedly triggering the same checkpoint also did not incorrectly increase progress.

The test demonstrated that checkpoint validation was based on the required order rather than simply counting checkpoint triggers.

5.13 MULTI-LAP RACE RESULTS

The lap progression system was evaluated using three checkpoints as one complete lap. After a vehicle successfully passed CP1, CP2, and CP3, its lap count increased and the next expected checkpoint was reset to CP1.

Each vehicle maintained its own current lap, next expected checkpoint, and race progress. This allowed connected players to progress independently during the same race.

The racing module was configured for five laps, and race completion was detected after the required lap progression was reached.



Figure 5.8 Multi-lap race progression with individual player race states.

5.14 OVERALL MULTIPLAYER PLATFORM RESULTS

The complete platform was evaluated through the common multiplayer workflow and the individual gameplay operations of both game modes.

The 2D platformer test covered authentication, session creation and joining, lobby participation, player spawning, ownership, movement synchronization, elimination, and leaderboard functionality.

The 3D racing test covered networked vehicle spawning, ownership assignment, client input transmission, server-authoritative vehicle movement, transform synchronization, countdown, checkpoint validation, and multi-lap progression.
The results demonstrate that the platform supports two different multiplayer game modes through a common connection and networking framework. The common layer manages player connection and networked objects, while each game mode handles its own gameplay operations.

The current racing implementation includes race completion detection but does not include final finishing-position calculation, racing leaderboard integration, car selection, or track selection.

5.15 LIMITATIONS

The current implementation demonstrates the integration of common multiplayer services with two game modes. However, some additional features remain outside the scope of the developed prototype.

- The racing module does not currently calculate final finishing positions.
- Racing-specific leaderboard integration is not included.
- Car selection and track selection are not implemented.
- Advanced lobby features such as ready states, matchmaking, and private rooms remain future enhancements.
- The platform has not been evaluated through large-scale testing with a high number of concurrent players.
- Further testing is required to study performance, latency, and scalability under different network conditions.
- Player profiles, persistent statistics, and achievements are not included in the current implementation.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project developed a reusable real-time multiplayer gaming platform using Unity, Unity Gaming Services, and Netcode for GameObjects. The platform integrates player authentication, session creation and joining, lobby management, networked object spawning, ownership, input handling, and multiplayer synchronization within a common framework.

The implementation supports two game modes: a 2D multiplayer platformer and a 3D multiplayer racing game. The platformer demonstrates networked player movement, lobby functionality, player elimination, and leaderboard operations. The racing module extends the platform with networked vehicle spawning, ownership assignment, RPC-based input transmission, server-authoritative vehicle movement, synchronized countdown, ordered checkpoint validation, and five-lap race progression.

Testing with multiple connected game instances demonstrated the integration of the common multiplayer workflow with the gameplay-specific components of both modes. The implementation establishes a foundation through which additional multiplayer games can be developed using the existing authentication, session management, and networking structure.

Overall, the project demonstrates how a common multiplayer framework can support different gameplay requirements while keeping the connection and networking components separate from game-specific logic.

6.2 FUTURE WORK

The developed platform can be extended to provide a more complete and scalable multiplayer gaming environment. Future development can focus on introducing additional levels, gameplay variations, and power-ups to provide greater game variety and player interaction.

The common multiplayer architecture can be reused to develop additional mini-games, allowing different games to operate using the existing authentication, session management, lobby, and networking framework. The lobby can also be enhanced with player lists, ready states, matchmaking, and private game rooms.

For the 3D racing module, future improvements can include lap-progress display, final finishing-position calculation, racing-specific leaderboard functionality, car selection, and track selection.

Additional platform features may include player profiles, persistent statistics, achievements, and improved handling of player disconnections and reconnections. Large-scale multiplayer testing can also be conducted with more concurrent players to study network synchronization, latency, and scalability.

Cross-platform support can be explored to allow players using different supported devices to participate in the same multiplayer environment.

REFERENCES

[1] [1] L. Gautier and C. Diot, “Design and evaluation of MiMaze, a multi-player game on the Internet,” in *Proc. 3rd IEEE Multimedia Computing and Systems (MMCS)*, 1998, pp. 233–236, doi: 10.1109/MMCS.1998.693647.

[2] [2] Y.-J. Lin, K. Guo, and S. Paul, “Sync-MS: Synchronized messaging service for real-time multi-player distributed games,” in *Proc. 10th IEEE Int. Conf. Network Protocols (ICNP)*, 2002, pp. 155–164, doi: 10.1109/ICNP.2002.1181396.

[3] [3] X. Jiang, F. Safaei, and P. Boustead, “Latency and scalability: A survey of issues and techniques for supporting networked games,” in *Proc. 13th IEEE Int. Conf. Networks (ICON)*, 2005, vol. 1, doi: 10.1109/ICON.2005.1635458.

[4] [4] K. S. Moon, V. Muthukkumarasamy, A. T.-A. Nguyen, and H. S. Kim, “Maintaining consistency in distributed network games,” in *Proc. 13th IEEE Int. Conf. Networks (ICON)*, 2005, vol. 1, doi: 10.1109/ICON.2005.1635506.

[5] [5] M. Mauve, J. Vogel, V. Hilt, and W. Effelsberg, “Local-lag and timewarp: Providing consistency for replicated continuous applications,” *IEEE Trans. Multimedia*, vol. 6, no. 1, pp. 47–57, Feb. 2004, doi: 10.1109/TMM.2003.819751.

[6] [6] L. Pantel and L. C. Wolf, “On the impact of delay on real-time multiplayer games,” in *Proc. 12th Int. Workshop Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2002, pp. 23–29, doi: 10.1145/507670.507674.

[7] [7] Y. W. Ahn, A. M. K. Cheng, J. Baek, and P. S. Fisher, “A multiplayer real-time game protocol architecture for reducing network latency,” *IEEE Trans. Consumer Electron.*, vol. 55, no. 4, pp. 1883–1889, Nov. 2009, doi: 10.1109/TCE.2009.5373746.

[8] [8] A. Bharambe, J. Pang, and S. Seshan, “Colyseus: A distributed architecture for online multiplayer games,” in *Proc. 3rd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, San Jose, CA, USA, 2006, pp. 155–168.

[9] [9] S. Fujita, “Load balancing of peer-to-peer MMORPG systems with hierarchical area-of-interest management,” *Int. J. Networked and Distributed Computing*, vol. 3, no. 3, pp. 177–184, Aug. 2015, doi: 10.2991/ijndc.2015.3.3.5.

[10] [10] S. Liu, M. Claypool, A. Kuwahara, J. Scovell, and J. Sherman, “The effects of network latency on competitive first-person shooter game players,” in *Proc. 13th Int. Conf. Quality of Multimedia Experience (QoMEX)*, 2021, pp. 151–156, doi: 10.1109/QoMEX51781.2021.9465419.